

Snooker2D 项目中期总结报告

井淳，鲁亦智，何广一 台球组

2026 年 7 月 14 日

1 项目概况

Snooker2D 是一个基于 C++17 和 Qt 6 的 2D 斯诺克台球游戏，采用课程讲授的 MVVM 五层架构。项目三人协作，10 天工期，当前处于中期检查节点。

2 技术架构

2.1 分层设计

项目拆分为四层编译目标，App 层在主可执行文件中完成装配：

层	职责	编译依赖
Common	全局类型、常量、工具函数、DTO	无
Model	游戏状态机、物理引擎、规则判定	Common + Qt::Core
ViewModel	数据翻译、状态推送、命令处理	Model + Common + Qt::Core
View	界面渲染、用户输入	Common + Qt::Widgets
App	对象装配、信号槽绑定	所有层

关键约束：View 层不链接 ViewModel/Model（CMake 编译期强制）。

2.2 层间通信

ViewModel 和 View 的跨层信号槽由 App 层在 App.cpp 中集中绑定（10 条 connect）。Model 到 ViewModel 的通知绑定在 ViewModel 构造函数内部完成。

绑定类型	方向	连接位置	数量
属性绑定	ViewModel → View	App.cpp	5
命令绑定	View → ViewModel	App.cpp	5
通知绑定	Model → ViewModel	ViewModel 构造	9

3 已完成功能

- 斯诺克标准 22 颗球的初始摆球和球桌渲染
- 鼠标瞄准与击球（含球杆动画）

- 力度/角度双模控制（滑块 + 鼠标拖动）
- 球-球弹性碰撞、球-库边反弹、摩擦力减速
- 袋口检测与落袋、D 区白球复位
- 计分板、游戏信息面板、比赛重启
- 60 条 Common 层单元测试、架构护栏自动检查

4 分工完成情况

4.1 开发者 A —View 层

负责球桌渲染、鼠标交互、球杆动画、控件布局。

球桌渲染：GameView::paintEvent 中实现台面（绿色）、库边（棕色）、6 袋口（黑色）、22 颗球（按类型着色 + 高光效果）的完整绘制管线。通过 gameToPixel 实现游戏坐标到像素坐标的转换，支持居中坐标系。

鼠标交互：将击球操作从按钮改为鼠标操作——移动调整瞄准方向、滚轮调节力度、按下触发球杆动画、松开进入物理击球。GameInfoPanel 改为 bind 模式接收 GameInfoViewState 更新。

技术难点：坐标系转换（居中坐标系与像素坐标的自动检测）、球杆动画与鼠标事件的时序配合。

4.2 开发者 B —Model + ViewModel 层

负责游戏状态机、物理引擎、规则判定、数据翻译和状态推送。

物理引擎：固定时间步长（1/60s）的六步模拟流程——运动、球-球碰撞、球-库边碰撞、袋口检测、摩擦减速、硬边界约束。球-球碰撞采用等质量弹性碰撞公式，库边碰撞通过 closestPointOnSegment 计算接触点法线。

规则引擎：FoulResult 结构体承载犯规类型、描述文本和罚分。checkFoul 按优先级检测白球落袋、空杆、先击错球、无球碰库四种犯规。

GameSessionViewModel：唯一的 ViewModel 协调器。构造时连接 Model 的 9 个信号到内部处理槽。5 个 pushXxxState() 方法从 Model 读取数据、构造 ViewState DTO、emit 到 View。5 个 public slots 接收 View 命令。

技术难点：物理参数的经验性调优（恢复系数 0.96→0.88、库边弹性 0.77 的多轮试错）、库边碰撞中 closestPointOnSegment 的应用。

4.3 开发者 C —Common + App 层

负责公共类型与工具、DTO 设计、应用装配、构建配置和项目管理。

4.3.1 Common 层

Constants.h: 球桌尺寸 (800×400)、球参数 (半径 10、质量 1.0、红球 15、总计 22)、袋口半径 (16)、D 区几何、物理参数 (步长 1/60s、摩擦 0.99、碰撞恢复 0.88、库边弹性 0.77、速度阈值 0.05)、游戏参数 (最大力度 100、最大初速 15.0) 和应用配置 (窗口标题、尺寸)。

Types.h: Vector2D 结构体——14 个运算符 (+/-/一元-/ * / / / += / -= / *= / /= / == / != / dot / cross / length / lengthSquared) 成员函数委托给 MathUtils 自由函数。4 个枚举 (BallType 8 种球、FoulType 7 种犯规、GamePhase 6 阶段、PlayerId)。3 个 inline constexpr 辅助 (ballValue、isColorBall、oppositePlayer)。

MathUtils.h/.cpp: 9 个函数 (dot/cross/length/lengthSquared/normalize/reflect/distance/circleOverlap) 只实现当前被调用的函数，不做预铺。

GameViewState.h: 5 个纯数据 DTO——BallViewState、TableViewState、CueViewState、ScoreViewState、GameInfoViewState。DTO 中 canAim/canShoot/canShoot 等布尔字段使 View 无需解析游戏规则即可控制交互状态；phaseKind 枚举字段供 View 做样式分支而不依赖中文字符串。

设计决策：FoulResult 结构体定义在 Model 层 Rules.h (含 QString 依赖 Qt，不适合纯 C++ 的 Common 层)。袋口坐标由 Model 层 Table.h 管理。

4.3.2 App 层

App.h/.cpp: 唯一的装配车间。App::run() 中创建 GameSessionViewModel 和 MainWindow，通过 10 条 QObject::connect 集中完成跨层信号槽绑定——属性绑定 5 条 (ViewModel 信号 → View 槽) 和命令绑定 5 条 (View 信号 → ViewModel 槽)。MainWindow 通过 4 个 getter 暴露子 View 供 App 绑定。通知绑定 (Model → ViewModel) 保留在 GameSessionViewModel 构造函数内部，因 ViewModel 本就持有 Model 指针。

MainWindow: 纯布局容器。setupUI() 创建 4 个 View 控件并排列左右分栏，4 个 getter 暴露子 View 指针。不含任何跨层连接逻辑。

架构演进: App 层经历了三轮重构。第一轮：引入 GameUiBus 作为通信中间件，View 绑定 Bus 而非具体 ViewModel 类型。第二轮：删除 3 个子 ViewModel (GameViewModel/CueControlViewModel/ScoreViewModel) 全部合并到 GameSessionViewModel。第三轮：取消 Bus，各层自管 signal/slot，App 集中 connect。每次重构都是团队讨论后共同推进，当前方案是三轮迭代后的收敛结果。

4.3.3 构建配置与测试

CMake 四层 target (common/model/viewmodel/view)，View 层编译期不链接 ViewModel/Model。架构护栏脚本 check_mvvm_boundaries.ps1 (含 7 项检查) 作为 CMake custom target 持续验证依赖边界。60 条 Common 层单元测试 (向量运算 15 条、几何 6 条、Vector2D 运算符 15 条、成员函数 6 条、Types 辅助 12 条、常量 6 条) 全部通过，无第三方测试框架依赖。

4.3.4 协作经历

与 B 在 Common 层的接口协调：B 在物理调优时直接修改 Constants.h 参数 (摩擦、弹性系数) 并新增 closestPointOnSegment 函数。C 在后续整理中接受并统一了这些变更——分层架

构中的灰色地带需要灵活处理。与 A 在 MainWindow 的协作中，信号连接经历了按钮击球 → 鼠标击球 →Bus→ 直接绑定的多次变迁，每次都需要精确协调接口变更。

5 团队协作

三位成员通过 Git 协作，提交遵循 `type(layer): description` 规范。
提交统计：

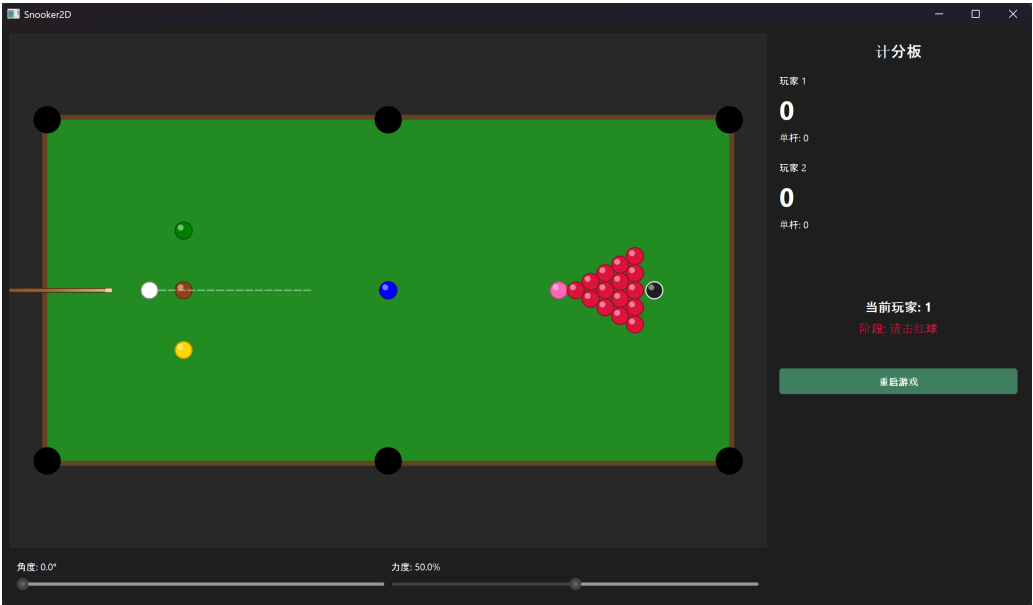
开发者	提交数	负责层
LOnion1124 (A)	14	View
Ironhammer Std (B)	8	Model / ViewModel
Ventub (C)	13	Common / App / 文档

协作亮点：MVVM 分层开发使三人真正做到并行工作——View 层在 Model 未完整时即可通过 DTO 骨架进行 UI 开发。跨层接口变更通过 Git 历史可追踪，未出现编译失败级别的接口不匹配。

架构演进：项目经历了三轮架构重构——引入 contracts 层和 GameUiBus、删除子 View-Model 合并到 GameSessionViewModel、取消 Bus 改用 App 集中绑定。每次重构都是团队共同决策、逐层推进。

6 阶段性成果展示

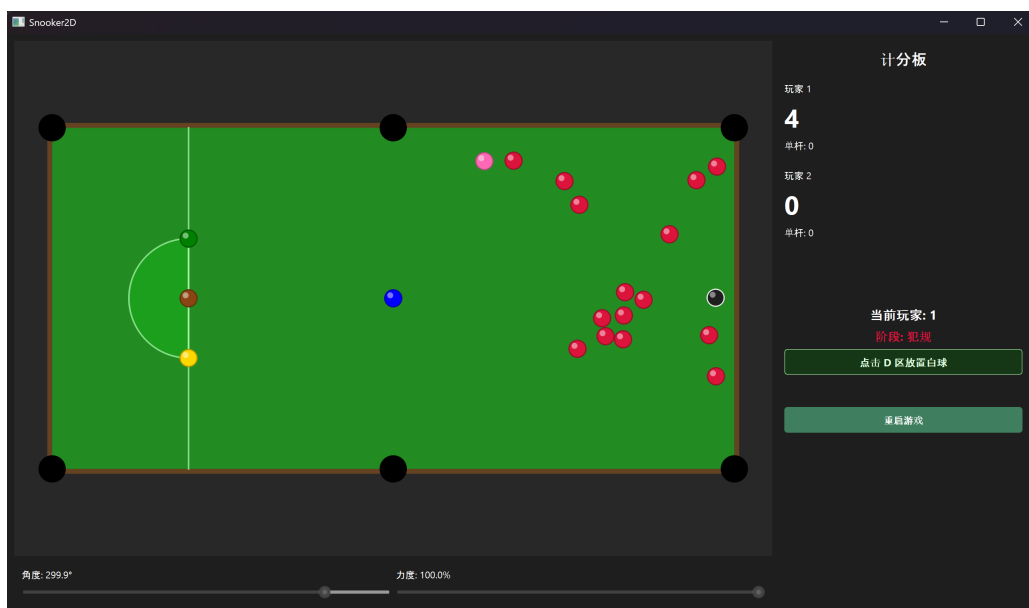
截图 1：游戏主窗口



截图 2：物理碰撞效果



截图 3：D 区白球复位



7 个人反思与收获

7.1 开发者 A

View 层开发让我深刻体会到 MVVM 架构中“数据驱动界面”的含义——所有绘制都来自 ViewModel 的属性变更通知，不再手动管理刷新。鼠标交互的实现比预期复杂，从按钮切换到鼠标涉及角度计算、坐标转换、动画衔接、事件过滤等多个细节。与 C 的协作中，App 层的信

号连接经历了多次重构，每次都需要精确调整，让我理解了集成层的维护成本。

7.2 开发者 B

Model 层开发最核心的挑战不在于物理公式本身，而在于处理“不完美的现实”——球碰撞恢复系数、库边不同弹性、摩擦力非线性，这些细节在理论推导中被简化，但在视觉反馈中会暴露。多次参数试错让我理解了物理引擎调优需要耐心。ViewModel 层的设计让我体会到接口设计的权衡：暴露太少 View 无法工作，暴露太多则性能和维护成本上升。与 A 和 C 的跨层协作中，最大的收获是“约定优于实现”——只要层间信号签名约定清楚，三方可以真正并行。

7.3 开发者 C

作为 Common 层和 App 层的负责人，我承担了“基础设施提供者”和“装配工人”的双重角色。Common 层要求对需求的预判——Vector2D 的复合赋值运算符在一开始看似多余，但当物理引擎要求就地更新速度向量时立刻体现价值。App 层要求对所有层接口的全局理解——每一条 connect 都意味着理解一个信号的来源和去向。

与 B 的协作让我认识到，分层架构中的灰色地带需要灵活处理——B 在物理调优时修改了 Common 层文件，严格来说超出了分工约定，但为了迭代效率我选择了接受而非回退。三轮架构重构（Bus 引入 → 子 VM 合并 → Bus 取消）让我切身体会了架构设计的权衡——没有一劳永逸的方案，只有在实践中持续迭代的改进。

8 智能体的使用

本项目选择以人为主的开发模式。大模型智能体在开发中仅用于补充性工作：代码补全与格式化、文档草稿生成、构建配置调试建议。所有核心架构设计、接口协调、技术决策均由团队成员自主完成。

9 后续计划

1. 完善斯诺克规则引擎（犯规判定全部场景、自由球、输赢判定）
2. 补充 Model 层单元测试和端到端集成测试
3. CI/CD 配置
4. UI 美化与资源整理
5. 文档完善